

Notre application peut rencontrer des situations exceptionnelles susceptibles d'empêcher son exécution normale — par exemple, des données incorrectes ou des conditions imprévues.

Pour garantir une exécution fiable et éviter l'arrêt brutal du programme, nous allons mettre en place un mécanisme de gestion des exceptions personnalisées.

☒ **Instruction 32 : Simplification de la méthode addAnimal**

La méthode `addAnimal(Animal animal)` retourne actuellement un `booléen` indiquant si l'ajout a réussi.

Or, ce mécanisme n'est pas toujours adapté à la gestion d'erreurs complexes.

1. Modifiez le type de retour de la méthode addAnimal :

→ Passez de `boolean` à `void`.

2. Supprimez les tests internes qui vérifient si le zoo est plein.

Ces vérifications seront désormais remplacées par une exception dédiée, afin de séparer clairement la logique de traitement des erreurs.

3. Assurez-vous que la méthode continue d'ajouter les animaux valides normalement, sans renvoyer de valeur.

☒ **Instruction 33 : Création d'une exception personnalisée ZooFullException**

Lorsqu'un zoo atteint sa capacité maximale, il ne doit plus accepter de nouveaux animaux, mais sans interrompre le programme brutalement.

1. Créez une classe d'exception personnalisée ZooFullException qui hérite d'Exception.

Cette classe doit contenir un message explicatif (par exemple : "Le zoo est plein, impossible d'ajouter un nouvel animal.").

2. **Modifiez la méthode `addAnimal`** pour qu'elle lève cette exception dès que le nombre maximum de cages est atteint.

3. **Dans la classe `main` :**

- Modifiez la méthode `main` de manière que le programme poursuive normalement son exécution même lorsque la méthode `addAnimal` rencontre une situation anormale
- Continuez à afficher le nombre total d'animaux après chaque tentative d'ajout (qu'elle réussisse ou échoue).
- Réduisez le nombre de cages à 3 dans le constructeur du zoo et testez votre code pour vérifier le comportement lorsque la capacité est dépassée.

☒ **Instruction 34 : Validation des données – âge de l'animal**

Un animal ne peut pas avoir un âge négatif. Cette situation doit être considérée comme une erreur de saisie.

1. **Créez une nouvelle exception personnalisée `InvalidAgeException`** (héritant également d'`Exception`).

Le message pourrait être : "Âge d'animal invalide : l'âge ne peut pas être négatif."

2. **Dans la méthode `addAnimal`**, ajoutez une vérification avant l'ajout :

Si `animal.getAge() < 0`, levez une `InvalidAgeException`.

Sinon, ajoutez l'animal normalement.

3. **Dans la méthode `main` :**

- Gérez cette exception
- Affichez un message d'erreur explicite si un âge invalide est détecté.

- Continuez à afficher le nombre d'animaux présents après chaque tentative.

4. Testez votre programme avec :

- Un animal valide,
- Un animal avec un âge négatif,
- Et plusieurs ajouts jusqu'à atteindre la capacité maximale du zoo.